

Redspher

Full Stack Developer

Technikai interju felkeszito - mobil PDF

React + TypeScript	Node.js + REST
AWS + CI/CD	AI / MCP / Agents

Cel

Ez a verzio mar konkretan technikai interjura keszit fel: valaszvazak, trade-offok, mini system design, kodolasi es architekturaskerdesek.

Hogyan hasznald

- Eloszor olvasd vegig a Prioritas oldalt.
- Utana gyakorold hangosan a kerdes-valasz kartyakat.
- A system design reszeknel 2-3 percben mondd el az architekturat.
- A saját tapasztalataidbol mindig hozz egy konkret peldat.

Forras: a feltoltott Redspher job description, plusz official MCP es AWS dokumentacio a relevans technikai reszekhez.

Tartalom

- 1. Mit fognak valószínűleg merni?
- 2. 60 másodperces technikai bemutatkozás
- 3. TypeScript és React kérdések
- 4. Node.js, REST API, error handling
- 5. AWS, CI/CD, observability
- 6. AI / MCP / agent alapú megoldások
- 7. Mini system design szcenariók
- 8. Legacy PHP integráció
- 9. Kerdezz vissza okosan
- 10. Utolsó napi ismétlés

JD-ból következő technikai fókusz

- Business-oriented full-stack fejlesztés.
- React + TypeScript frontend és backend services.
- AWS cloud környezet, REST APIs, Git, CI/CD.
- AI-driven solutions: MCP, AI agents, orchestration, automation.
- PHP előny lehet legacy karbantartás vagy integráció miatt.

1. Mit fognak valószínűleg merni?

Nem csak lexikont keresnek

A JD alapján a lenyeg: tudsz-e business igényből működő, karbantartható full-stack megoldást építeni, és tudsz-e értelmesen gondolkodni új AI/automation irányokról.

Tema	Mit bizonyíts
React/TS	Tiszta komponensek, state, effects, performance, type safety.
Backend	REST, validation, auth, hibakezeles, async folyamatok.
AWS	Pragmatikus deployment, IAM, monitoring, managed services.
Architecture	Trade-offok: gyors szállítás vs skála, egyszerűség vs rugalmasság.
AI/MCP	Nem hype: biztonság, tool calling, audit, human approval, evals.
Business	Logistics/pricing domain: pontosság, latency, adatminőség, audit.

Legjobb pozícionálás

Full-stack fejlesztő vagyok erős React/TypeScript háttérrel, de a backend, integrációk, observability és automation oldal sem idegen. A technológiát nem önmagáért, hanem business probléma megoldására választom.

2. 60 masodperces technikai bemutatkozás

Ezt ne olvasd fel szó szerint. Tanuld meg a szerkezetet, és mondd el természetesen.

Valaszvaz

I am a full-stack developer with strong React and TypeScript experience, and I have also worked on Node.js based backend and real-time systems. In my previous roles I built business-facing applications, improved frontend architecture, integrated APIs, worked with monitoring and release processes, and created automation around internal workflows. What I like about this role is that it combines practical full-stack product work with AWS and new AI/automation topics. I am especially interested in building useful AI-assisted workflows, but with proper security, observability and human control.

Ha rakerdeznek az AWS-re

I am not presenting myself as a senior AWS architect, but I have hands-on AWS learning and project experience, and I understand the main building blocks: Lambda/API Gateway, IAM, S3, DynamoDB/RDS trade-offs, queues, CloudWatch, and deployment basics. I am comfortable learning the exact company setup quickly.

Ha rakerdeznek az AI-ra

I would keep it pragmatic: identify a narrow business workflow, connect the model to safe tools and relevant context, add guardrails, logging, human approval for risky actions, and measure quality before scaling it.

3. TypeScript - amit tudnod kell

Fo uzenet

A TypeScript nem csak 'JS type-okkal'. Interjun azt varjak, hogy type safety-t hasznalj a runtime hibak csökkentesere, API szerzodesek tisztazasara es refactor biztonsagra.

TS

Mi a kulonbseg any es unknown kozott?

any kikapcsolja a type checkinget, unknown viszont biztonságos: elobb narrowolni kell, mielőtt hasznalod. Kulso API, JSON parse vagy user input eseten unknown + validation jobb.

TS

Mikor használnal union type-ot?

Amikor egy érték több, jól definiált állapotban lehet. Pelda: loading | success | error. Discriminated unionnel a UI állapotok sokkal biztonságosabban kezelhetők.

TS

Mi a generics lenyege?

Ujrahasznalhato, de type-safe kod. Pelda: egy API client visszateresi tipusat parameterkent adom meg, így a caller oldalon pontos typing marad.

TS

Hogyan kezelned a backend DTO-kat TypeScriptben?

Kulon request/response DTO, runtime schema validation, es ahol lehet shared contract vagy OpenAPI generalt tipusok. Fontos: TypeScript compile-time, nem validalja a runtime inputot.

```
type ApiState<T> = | { status: 'loading' } | { status: 'success'; data: T } | { status: 'error'; error: string }; function render<T>(state: ApiState<T>) { if (state.status === 'success') return state.data; if (state.status === 'error') return state.error; return 'Loading...'; }
```

4. React - valoszinu kerdesek

Fo uzenet

Ne csak hook neveket sorolj. Mutasd, hogy tudod: mi legyen local state, server state, derived state, es mikor kell performance optimalizacio.

REACT

Mikor hasznalnal useEffectet?

Side effectre: adatbetoltes inditasa, subscription, DOM-on kivuli integracio. Nem hasznalnam egyszeru derived state szamitasra, mert az renderbol vagy useMemoval kezelheto.

REACT

Mi a controlled component?

Az input erteket React state irányítja. Elonye: validacio, conditional UI, submit logika tisztabb. Hatranya: nagy formoknal performance es boilerplate.

REACT

Mikor kell memo/useMemo/useCallback?

Ha merheto ujrarenderelesi vagy referencialis stabilitasi problema van. Nem default mindenre, mert komplexebbe teszi a kodot.

REACT

Hogyan kezelled a server state-et?

Cache, loading/error state, refetch, invalidation. Nagy appban erre jo egy dedikalt library vagy centralizalt adatlekeresi minta, nem csak ad-hoc useEffect.

REACT

Hogyan optimalizalnal egy lassu React oldalt?

Meres eloszor: React Profiler, network waterfall, bundle size. Utana: memoization, virtualizalt lista, code splitting, kevesebb global state, API payload csokkentese.

REACT

Mire figyelsz komponens designnal?

Kis felelosseg, explicit props, reusable UI komponensek, business logika elkulonitese, tesztelhető es olvashato struktura.

React mini gyakorlati feladatok

1. API tabla filterrel

- State: query, filters, pagination, selected row.
- Server state: loading/error/data, cache, refetch.
- UX: debounced search, empty state, retry, skeleton.
- Types: DTO -> UI model mapping.

2. Pricing form

- Controlled vagy form library, schema validation.
- Backend validaciohoz igazodo error map.
- Optimistic UI csak akkor, ha rollback egyszeru.
- Money/currency/rounding kezelese nagyon fontos.

3. Realtime status dashboard

- Polling, SSE vagy WebSocket trade-off.
- Reconnect strategy, stale data jelzes.
- Backpressure: ne renderelj minden eventet azonnal.
- Observability: client error tracking, latency metrics.

Sajat peldad

Erdemes behozni a real-time/WebSocket es React/TypeScript tapasztalatot: hogyan kezeltel live adatot, release-t, monitoringot, vagy API integraciot production kornyezetben.

5. Node.js backend

Fo uzenet

A Node eros I/O-bound backendekhez, API-khoz es integraciokhoz. Interjun a lenyeg: event loop, async hibakezeles, struktura, validacio, observability.

NODE

Hogyan mukodik roviden a Node event loop?

Single-threaded JS execution, de I/O muveletek libuv/onallo OS mechanizmusokon keresztül aszinkron futnak. CPU-heavy feladat blokkolhat, arra worker thread, queue vagy kulon service kell.

NODE

Hogyan strukturalisz egy backend service-t?

Controller csak HTTP input/output, service business logika, repository/adapters kulso rendszerekhez, validation layer, central error handling, config es logging kulon.

NODE

Hogyan kezelned az async hibakat?

try/catch async fuggvényekben, central error middleware, typed/domain errorok, strukturalt log, correlation id, es kulso API-khoz timeout/retry/circuit breaker ha indokolt.

NODE

Mikor használnal queue-t?

Ha a folyamat lassu, megbizhatatlan kulso rendszerre var, ujraprobalhato, vagy nem kell szinkron valasz. Pelda: pricing recalculation, email, import/export, AI batch analysis.

NODE

Hogyan veded a Node API-t?

AuthN/AuthZ, input validation, rate limit, CORS helyesen, secrets manager, dependency scanning, output sanitization, least privilege IAM, audit log fontos muveletekre.

```
// Express-style central error idea app.use((err, req, res, next) => {
  logger.error({ err, requestId: req.id }); if (err.kind ===
  'ValidationError') return res.status(400).json({ code: 'BAD_REQUEST'
  }); return res.status(500).json({ code: 'INTERNAL_ERROR' }); });
```

6. REST API design

Fo uzenet

REST-nel az interju nem definicio-vadaszat. A lenyeg: tiszta resource modell, HTTP szemantika, hibak, auth, idempotencia, pagination es evolvability.

Tema	Jo valasz
Status codes	200/201/204, 400 validation, 401 auth, 403 permission, 404, 409 conflict, 422 domain validation, 429 rate limit, 5xx server.
Idempotencia	GET/PUT/DELETE alapbol idempotens. POST-hoz idempotency key lehet fizetes/pricing/order muveletnel.
Pagination	Offset egyszeru, cursor stabilabb nagy/valtozo adathoz.
Versioning	URI vagy header. Fontosabb: backwards compatibility, deprecation, contract tests.
Errors	Stabil error code + message + details + request id. Ne leakelj stack trace-t.

REST

Hogyan terveznel shipment tracking API-t?

Resources: /shipments, /shipments/{id}, /shipments/{id}/events.
Listazashoz filters es cursor pagination. Eventek immutable modon tarolhatok, a current status szarmaztatott/denormalizalt lehet.

REST

Hogyan kezelned a breaking changeket?

Elkerulom, ha lehet additive changes-szel. Ha breaking kell: uj version, deprecation window, migration guide, monitoring, es consumer communication.

REST

Enum bovitese breaking change?

API consumer szempontbol lehet breaking, ha a kliens exhaustive switch-et hasznal es nem kezeli az ismeretlen ertekeket.
Dokumentacio es 'unknown fallback' kell.

7. AWS full-stack szinten

Ne próbálj túljatszani

Mondd, hogy az AWS-ben pragmatikus alapokkal és hands-on tanulással jössz. Full-stack interjú fontosabb a jó service választás, IAM/observability/release gondolkodás, mint minden szolgáltatás memoriter ismerete.

Use case	Valószínű AWS opció
Static frontend	S3 + CloudFront, CI/CD deploy, cache invalidation.
REST backend	Lambda + API Gateway kisebb servicehez, vagy ECS/Fargate tartós backendhez.
Async munka	SQS/EventBridge, DLQ, retry, idempotens worker.
Relációs adat	RDS/Aurora, migrations, backups, indexes.
Key-value / nagy skála	DynamoDB, partition key design, access patterns előre.
Monitoring	CloudWatch logs/metrics/alarms, tracing, dashboard, correlation id.
Secrets	Secrets Manager/SSM Parameter Store, ne env-ben hardcode.

Reliability talking points

- Timeout minden kulso hívásra.
- Retry exponential backoffal, de csak idempotens műveletnél.
- DLQ és alert, hogy a hibás üzenet ne vesszen el.
- Health check, alarms, rollback, runbook.
- Business KPI monitoring: pricing success rate, shipment event freshness, API latency.

AWS kerdes-valasz

AWS

Lambda vagy container?

Lambda jo event-driven, rovid, bursty workloadra. Container jobb hosszan futó processzre, custom runtime/dependency és stabil network kapcsolat esetén. Dont overengineer: a workload dont.

AWS

RDS vagy DynamoDB?

RDS jo komplex relációkhoz, SQL querykhez, tranzakciókhoz. DynamoDB jo nagy skalan, előre ismert access patternnel. Logistics/pricing domainben gyakran mindkettő lehet: transactional core + denormalizált read model.

AWS

Mit figyelsz IAM-nel?

Least privilege, role-alapu hozzáférés, külön env/account, secrets nem kódban, audit log. AI tooloknál különösen szűk scope kell.

AWS

Hogyan deployolnál biztonságosan?

Build-test-scan, staging, migration plan, canary/blue-green ahol indokolt, feature flag, health check, monitorozott rollout, gyors rollback.

AWS

Mit jelent observability?

Logs, metrics, traces együtt. Ne csak technikai metrika: request id, domain eventek, error rate, latency percentilis, business sikermutató.

8. Git es CI/CD

Interju valasz

A CI/CD lenyege nem az, hogy 'van pipeline', hanem hogy kicsi, reviewzott valtozasokat megbizhatoan, tesztekkel es rollback lehetoseggel tudsz productionbe juttatni.

Pipeline lepes	Mire jo
Lint/typecheck	Olcso hibak korai fogasa.
Unit tests	Business logika es komponensek gyors ellenorzese.
Integration tests	API, DB, kulso adapter szerzodesek.
Build artifact	Ugyanaz menjen stagingre es prodra.
Security scan	Dependency es secret hibak fogasa.
Deploy	Staging -> approval -> production.
Rollback	Verziozott artifact, feature flag, migration rollback terv.

Code reviewban ezt figyelem

- Megoldja-e a business problemat?
- Olvashato-e es tesztelhető-e?
- Van-e input validation es error handling?
- Nem okoz-e security vagy performance regressziót?
- Egyszeru-e visszavonni vagy monitorozni?

9. AI / MCP / agents - technikai szemmel

Fo uzenet

Itt az a jo, ha nem hype-olsz. Az ertek: AI-t biztonsagosan bekotni valos business workflow-ba, ahol vannak toolok, jogosultsag, audit, eval es emberi kontroll.

Fogalom	Rovid technikai magyarakzat
LLM	Szoveges/dontestamogato modell, nem determinisztikus business rule engine.
Agent	LLM + cel + eszkozhasznalat + tobb lepeses dontes/vegrehajtias.
Tool calling	A modell strukturalt parameterekkel kulso fuggvenyt/API-t hiv.
MCP	Standard protokoll LLM appok es kulso context/tool forrasok kozott.
RAG	Relevant dokumentum/adat betoltese valasz elott; nem ugyanaz, mint fine-tuning.
Orchestration	Lepesek, toolok, state, retry, approvals, audit osszefuzese.

MCP lenyeg

- Server features: resources, prompts, tools.
- Tools: model altal invokalhato funkcioik kulso rendszerekhez.
- Resources: context/adat, peldaul file, schema, domain adat.
- Security: input validation, access control, rate limit, output sanitization, user confirmation sensitive actionnel.

AI agent keres-vals

AI

Hogyan epitenel AI assistantot logistics csapatnak?

Narrow scope-pal kezdenem: pl. shipment/pricing adatok keresese es osszefoglalasa. RAG/context read-only modban, majd kesobb safe tool actions. Kell auth, audit, PII guardrail, human approval irasnal.

AI

Mi a veszelye az agenteknek?

Nem determinisztikus dontes, hallucination, prompt injection, jogosulatlan data access, tool misuse, adat exfiltracio. Ezert kell szuk jogosultsag, allowlistelt tool, approval, logging es eval.

AI

Hogyan mered egy AI workflow minoseget?

Golden test set, human review, precision/recall jellegu metrikak domain feladatra, latency/cost, fallback arany, user correction rate, audit of failed cases.

AI

Mikor nem hasznalnai AI-t?

Ha determinisztikus, szabalyalapu, olcso es auditkritikus folyamatrol van szo. Pelda: arszamitas core formula. AI inkabb asszisztens, javaslat, dokumentumkereses vagy exception triage.

AI

MCP tool biztonsag hogyan nez ki?

Validate input, scope permissions per user, rate limit, redact sensitive output, log tool calls, show user confirmation destructive vagy external side-effect action elott.

Jo mondat

I would start read-only, prove value, add observability and evaluation, then introduce controlled write actions with human approval.

10. Mini system design: logistics pricing

Feladat

Tervezd meg egy pricing tool architekturáját, ahol business felhasználók shipment paraméterek alapján arakat kapnak, és a rendszer auditolható.

Clarifying questions

- Real-time quote kell vagy batch pricing?
- Milyen adatok: distance, carrier, capacity, SLA, fuel surcharge?
- Ki módosít szabályokat és kell-e approval?
- Mennyi latency elfogadható?
- Kell-e audit log és explainability?

Javasolt architektúra

- React admin UI pricing formokkal és historyval.
- Node.js REST API auth + validation + pricing orchestration.
- RDS a rule/config/audit adatokhoz.
- SQS/EventBridge batch recalculation és external carrier sync miatt.
- CloudWatch metrics: quote latency, failure rate, stale data.
- AI csak asszisztensként: rule explanation, anomaly detection javaslat, nem core formula.

Trade-off

Ha gyorsan kell indulni: monolith/modular backend. Ha több domain csapat és nagy skála van: kesőbb service boundary pricing, shipment, carrier data menten.

11. Mini system design: shipment dashboard

Feladat

Epits dashboardot, amely megmutatja a szállitmanyok aktualis allapotat, keseseket es event historit.

Adatmodell

- Shipment: id, customer, route, status, ETA, carrier.
- ShipmentEvent: id, shipmentId, type, timestamp, source, payload.
- CurrentStatus lehet denormalizalt a gyors listazashoz.
- Eventek immutable modon tarolhatok audit miatt.

API otlet

- GET /shipments?status=&customer=&cursor=
- GET /shipments/{id}
- GET /shipments/{id}/events
- POST /shipments/{id}/notes vagy status override csak jogosultsaggal.
- Realtime: polling vagy SSE, ha csak server -> client update kell; WebSocket, ha ketiranyu interakcio is kell.

Buktatok

- Nagy lista virtualizalasa frontend oldalon.
- Cursor pagination offset helyett nagy valtozo adathoz.
- Idempotens event ingestion, duplicate handling.
- Stale data jelzese UX-ben.

12. Mini system design: AI operations assistant

Feladat

AI assistant, amely business user kérdésekre válaszol: 'Mely szállítványok vannak veszélyben?' vagy 'Mi okozta a pricing outlier?'

Biztonságos első verzió

- Read-only data access.
- User auth alapján szűrt adatok.
- RAG: policies, shipment metadata, pricing rule leírások.
- Toolok: searchShipments, getPricingAudit, summarizeDelayReasons.
- Human approval bármilyen write/export/email action előtt.

Architecture sketch

- React chat/workflow UI.
- Node orchestrator: prompt templates, tool registry, auth context.
- MCP serverek: logistics data resources + safe tools.
- Audit log: prompt metadata, tool calls, output, user approval.
- Evaluation set: gyakori kérdések és elvárt factual answers.

Jo trade-off mondat

For core operational decisions I would avoid autonomous write actions at first. I would start with explainable recommendations and keep a human in the loop.

13. Legacy PHP integracio

Miért kerülhet elő?

A JD szerint a PHP plusz pont legacy rendszerek karbantartasahoz vagy integralasahoz. Itt nem senior PHP tudast kell eljatszani, hanem jo migracios es integracios gondolkodast mutatni.

Strangler fig approach

- Legacy rendszert nem egyszerre cserelni.
- Eloszor API facade vagy adapter reteg.
- Uj funkcioik Node/React oldalon, legacy adat/folyamat fokozatos kivaltasa.
- Contract tests a regi es uj viselkedes kozott.
- Monitoring, rollback, feature flag.

LEGACY

Mit tennel, ha PHP legacy service-hez kell kapcsolodni?

Eloszor megertem az API/adat szerzodest, side effecteket es ownership-et. Adapter reteget epitek, input/output validacioval, logginggal es contract tesztekkel.

LEGACY

Mikor irnal ujra teljesen egy legacy modult?

Csak ha magas uzleti ertek, nagy karbantartasi fajdalom vagy biztonsagi kockazat indokolja. Egyebkent fokozatos kivaltas kisebb kockazat.

14. Kodolasi feladat - mire keszulj

Valoszinu formatum

Full-stack technikai interjun lehet kisebb React komponens, API endpoint, data transformation, validation, vagy pair programming jellegu feladat. A cel nem LeetCode, hanem tiszta gondolkodas.

Kodolasi checklist

- Kerj tisztazast input/outputra.
- Mondd ki a felteveseket.
- Eloszor egyszeru, mukodo megoldas.
- Utana edge case: null, empty, duplicate, invalid input.
- TypeScript type-ok es runtime validation kulon kezelese.
- Tesztelhető függvények, nem mindent UI-ba írni.
- Beszelj kozben: mit miért csinalsz.

Gyakorlati mini feladatok

- Shipment lista filterezese es rendezese TS-ben.
- React form validation pricing inputokra.
- REST endpoint: POST /quotes input validation + error response.
- Async worker: retry + idempotency key.
- API response normalizalasa UI modelle.

```
type QuoteRequest = { origin: string; destination: string; weightKg: number; serviceLevel: 'standard' | 'express'; }; function validateQuote(input: unknown): QuoteRequest { // runtime validation kell, nem eleg a TS type // productionben schema library vagy explicit validator throw new Error('implement'); }
```

15. Technikai sztorik saját tapasztalatból

Valassz 3 sztorit

Ne mindenre ugyanazt mondd. Legyen frontend, backend/real-time, automation/AI-AWS jellegu pelda.

Kerdes	Sajat sztori iranya
Performance	React/API payload, bundle, rendering, monitoring.
Scalability	Node/WebSocket, Redis pub/sub, horizontal scaling.
Business value	Automation, manual munka csökkentese, gyorsabb release vagy jobb visibility.
Failure	Hiba felismerese, rollback, monitoring, postmortem tanulsag.
Learning	AWS/AI training, demo projektek, uj stack gyors felszedese.

STAR sablon technikai valaszhoz

- Situation: milyen business/tech problema volt?
- Task: mi volt a te felelosseged?
- Action: milyen konkret technikai donteseket hoztal?
- Result: mi lett jobb? latency, stability, developer speed, business visibility.

16. Gyors kerdesbank - Frontend

FE

Mi a kulonbseg client state es server state kozott?

Client state UI-specifikus: modal, selected row. Server state kulso adat: cache, stale, refetch, errors, authorization.

FE

Hogyan kezelled a jogosultsagokat frontend oldalon?

UI-ban elrejtis/disable UX miatt, de security backend oldalon. Route guard csak kenyelmi reteg, nem biztonsagi hatar.

FE

Mitol lesz egy komponens ujrahasznalhato?

Jo absztrakcios szint, explicit props, business logika nelkuli UI primitive vagy jól izolált domain komponens.

FE

Hogyan kezelled a hibakat UI-ban?

Inline validation, toast csak nem kritikusra, retry, error boundary render hibakra, request id supportnak, user-friendly uzenet.

FE

Accessibility alapok?

Semantic HTML, keyboard navigation, focus management, label/input, contrast, aria csak ha semantic HTML nem eleg.

17. Gyors kerdesbank - Backend

BE

Mi a kulonbseg authentication es authorization kozott?

Authentication: ki vagy? Authorization: mit tehetsz? JWT/session csak az elso resz; permission check kulon domain logika.

BE

Hogyan kezelned a concurrency konfliktust?

Optimistic locking version fielddel, transaction, unique constraint, vagy idempotency key a domain szerint.

BE

Mikor kell transaction?

Ha tobb valtozasnak egyutt kell sikerulnie. Kulso API hivas transactionon belül rossz otlet lehet; saga/outbox kellhet.

BE

Mi az outbox pattern?

DB valtozas es event publish osszehangolasa: eventet ugyanabban a DB transactionben outbox tablaba irod, kulon publisher kuldi ki.

BE

Hogyan kezelned a kulso API hibakat?

Timeout, retry policy, circuit breaker ahol kell, fallback, clear error mapping, alerting es manual replay lehetoseg.

18. Gyors kerdesbank - Architecture

ARCH

Monolith vagy microservice?

Kezdesnek modular monolith gyakran gyorsabb es egyszerubb. Microservice akkor, ha kulon deployment, ownership, scaling vagy domain boundary indokolja.

ARCH

SQL vagy NoSQL?

Query/tranzakcio/adatkapcsolat alapjan dontok. SQL komplex relacios domainhez, NoSQL nagy skala es fix access pattern esetén.

ARCH

Sync vagy async kommunikacio?

Sync, ha azonnali valasz kell. Async, ha hosszu, ujraprobalhato, lazabb kapcsolasu folyamat, vagy external system megbizhatatlan.

ARCH

Hogyan tervezel skalahato listazast?

Indexek, filterek, cursor pagination, limit, stable ordering, projection, caching, read model ha kell.

ARCH

Hogyan kezelsz auditot?

Immutable event/audit log, actor, timestamp, old/new values, correlation id, retention, PII consideration.

19. Kerdezz vissza okosan

Technikai kérdések nekik

- A React/Node stackben mi a legnagyobb jelenlegi technikai fájdalom?
- Mennyire monolith vagy service-alapu a jelenlegi platform?
- AWS-ben milyen deployment modelt használtak: Lambda, ECS, EKS, mas?
- Milyen CI/CD es release folyamat van productionre?
- Van-e observability stack: logs, metrics, traces, error tracking?
- Az AI/MCP/agent iranyal van mar konkret use case, vagy exploration phase?
- A PHP legacy pontosan milyen szerepben maradt meg?
- Hogyan dolgozik egy fejlesztő a product/business stakeholderrel?

Jo benyomas

A kérdésekből látszódik, hogy nem csak technológiákat akarsz, hanem rendszer, ownership, release, business impact es maintainability is érdekel.

20. Utolso napi ismetles

90 perces plan

- 15 perc: technikai bemutakozas hangosan.
- 20 perc: React/TypeScript kerdesek.
- 20 perc: Node/REST/error handling.
- 15 perc: AWS/CI-CD/observability.
- 15 perc: AI/MCP/agent biztonsag.
- 5 perc: sajat kerdesek nekik.

Mondatok, amiket erdemes hasznalni

- I would start simple and add complexity only when the requirements justify it.
- I would validate input at the boundary and keep business logic separate from transport logic.
- For AI workflows, I would start read-only and add controlled actions later.
- I prefer measuring the problem first before optimizing blindly.
- I would design this around the business workflow, not around a fashionable architecture pattern.

Keruld

- Tul sok buzzword magyarazat nelkul.
- Mindent microservice-szel megoldani.
- AI-t core pricing/business rule helyett kontroll nelkul hasznalni.
- AWS szolgaltatasokat felsorolni trade-off nelkul.
- Azt mondani, hogy 'TypeScript miatt nincs runtime validation problema'.

Forrasok es megjegyzesek

Ez az anyag a feltoltott Redspher Full Stack Developer job description technikai elvarasaira epul: React, TypeScript, Node.js, AWS, REST APIs, Git/CI/CD, AI technologies, MCP, agents, automation, es opcionalis PHP legacy integracio.

MCP resz: Model Context Protocol official specification - server features, tools, resources, prompts, security considerations.

AWS resz: AWS Well-Architected Framework - reliability, operational excellence, observability es automation elvek.

Megjegyzes

A PDF nem hivatalos Redspher interjukerdes lista. Celja, hogy a JD-bol es hasonlo full-stack technikai interjukbol kovetkezo legvaloszinubb temakra keszitsen fel.